Name: Samir Kumar Gupta Roll No.: 2301730243 Environment: Ubuntu 24.04.5 LTS (OrbStack VM, arm64)

Question 6 - Deployment strategies and scaling

Question 6: Implement Kubernetes deployment strategies such as rolling updates and rollbacks, and demonstrate scaling of the application replicas.

Executed on an Ubuntu 24.04.5 LTS OrbStack machine (`ensp461-lab2`, arm64) with Minikube v1.39.0 and Kubernetes v1.37.0. Source, logs and screenshots: https://github.com/ksamirdev/ensp461-lab2.

Strategy

The Deployment uses `RollingUpdate` with `maxSurge: 1` and `maxUnavailable: 0`. Kubernetes starts one extra new pod, waits until its readiness probe passes, then removes one old pod, and repeats. Capacity never drops below 3 ready pods.

To measure this, `scripts/q6.sh` runs a probe loop during every rollout that calls `/health` through the NodePort about every 0.1 s and counts non-200 responses.

Rolling update 2.0.0 to 2.0.1

```
$ docker build -q --build-arg APP_VERSION=2.0.1 -f docker/Dockerfile -t localhost:5001/ensp461-
lab2:2.0.1 .
$ docker push --quiet localhost:5001/ensp461-lab2:2.0.1
$ kubectl set image deployment/lab2-web web=lab2-registry:5000/ensp461-lab2:2.0.1
$ kubectl annotate deployment/lab2-web kubernetes.io/change-cause='upgrade to 2.0.1' --overwrit
e
$ kubectl rollout status deployment/lab2-web --timeout=180s
Waiting for deployment "lab2-web" rollout to finish: 1 out of 3 new replicas have been update
d...
Waiting for deployment "lab2-web" rollout to finish: 2 out of 3 new replicas have been update
d...
Waiting for deployment "lab2-web" rollout to finish: 1 old replicas are pending termination...
deployment "lab2-web" successfully rolled out
probe: 61 ok, 0 failed
$ kubectl get rs -l app=lab2-web
NAME                  DESIRED   CURRENT   READY
lab2-web-644c5ccf89   3         3         3       # 2.0.1
lab2-web-fd84f8f84    0         0         0       # 2.0.0, kept for rollback
$ curl -s http://192.168.49.2:30080/health
{"status":"UP","version":"2.0.1"}
```

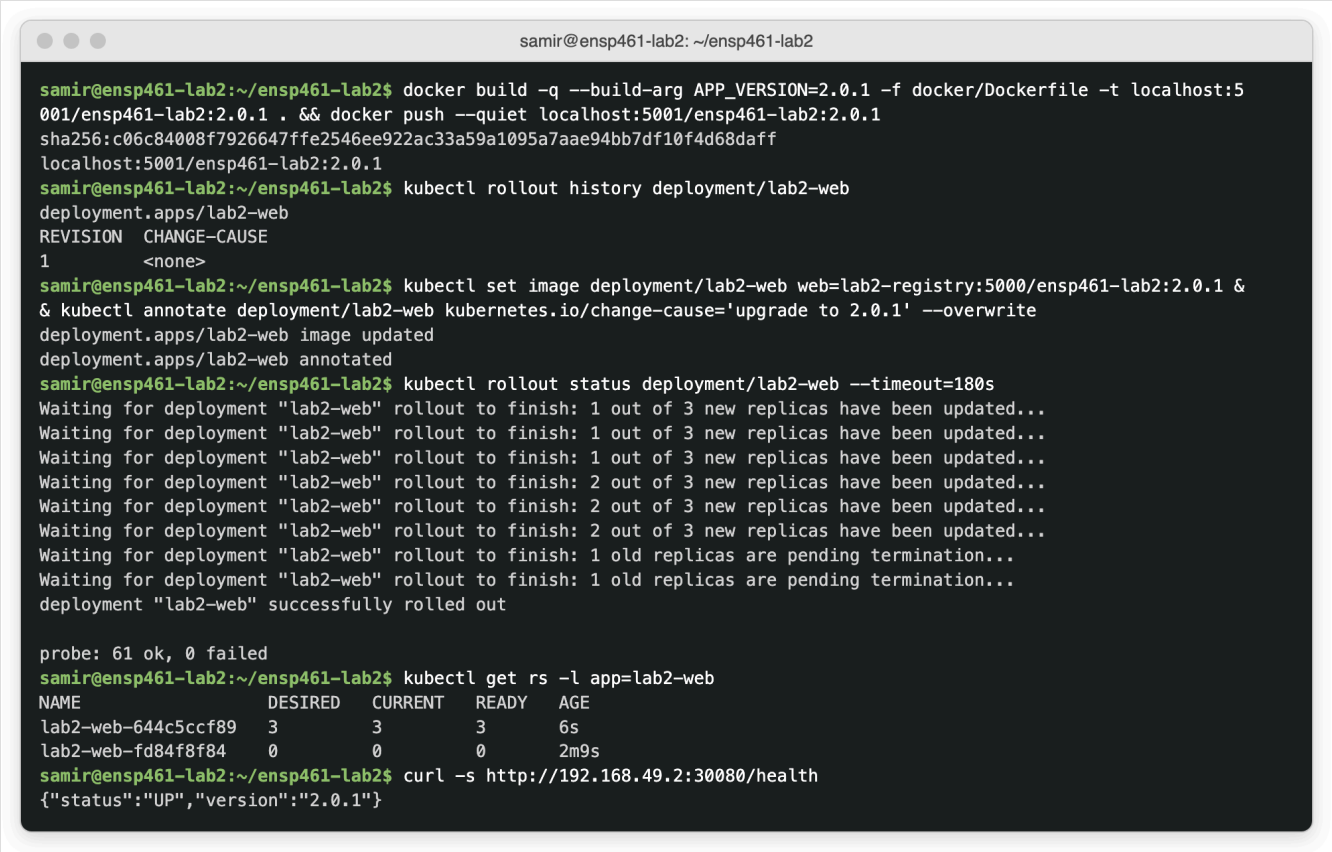


Fig: Rolling update to 2.0.1 with zero failed probe requests

Failed rollout and rollback

A bad release was simulated by pointing the Deployment at a tag that does not exist.

```
$ kubectl set image deployment/lab2-web web=lab2-registry:5000/ensp461-lab2:9.9.9-missing
$ kubectl rollout status deployment/lab2-web --timeout=30s
error: timed out waiting for the condition
$ kubectl get pods -l app=lab2-web
lab2-web-644c5ccf89-ggnxw   1/1     Running        0          31s
lab2-web-644c5ccf89-kwwts   1/1     Running        0          36s
lab2-web-644c5ccf89-md8zv   1/1     Running        0          35s
lab2-web-7bb9b46856-gcq7t   0/1     ErrImagePull   0          30s
$ curl -s http://192.168.49.2:30080/health
{"status":"UP","version":"2.0.1"}
$ kubectl rollout history deployment/lab2-web
REVISION  CHANGE-CAUSE
1         <none>
2         upgrade to 2.0.1
3         bad image
$ kubectl rollout undo deployment/lab2-web --to-revision=1
$ kubectl rollout status deployment/lab2-web --timeout=180s
deployment "lab2-web" successfully rolled out
probe: 30 ok, 0 failed
$ curl -s http://192.168.49.2:30080/health
{"status":"UP","version":"2.0.0"}
```

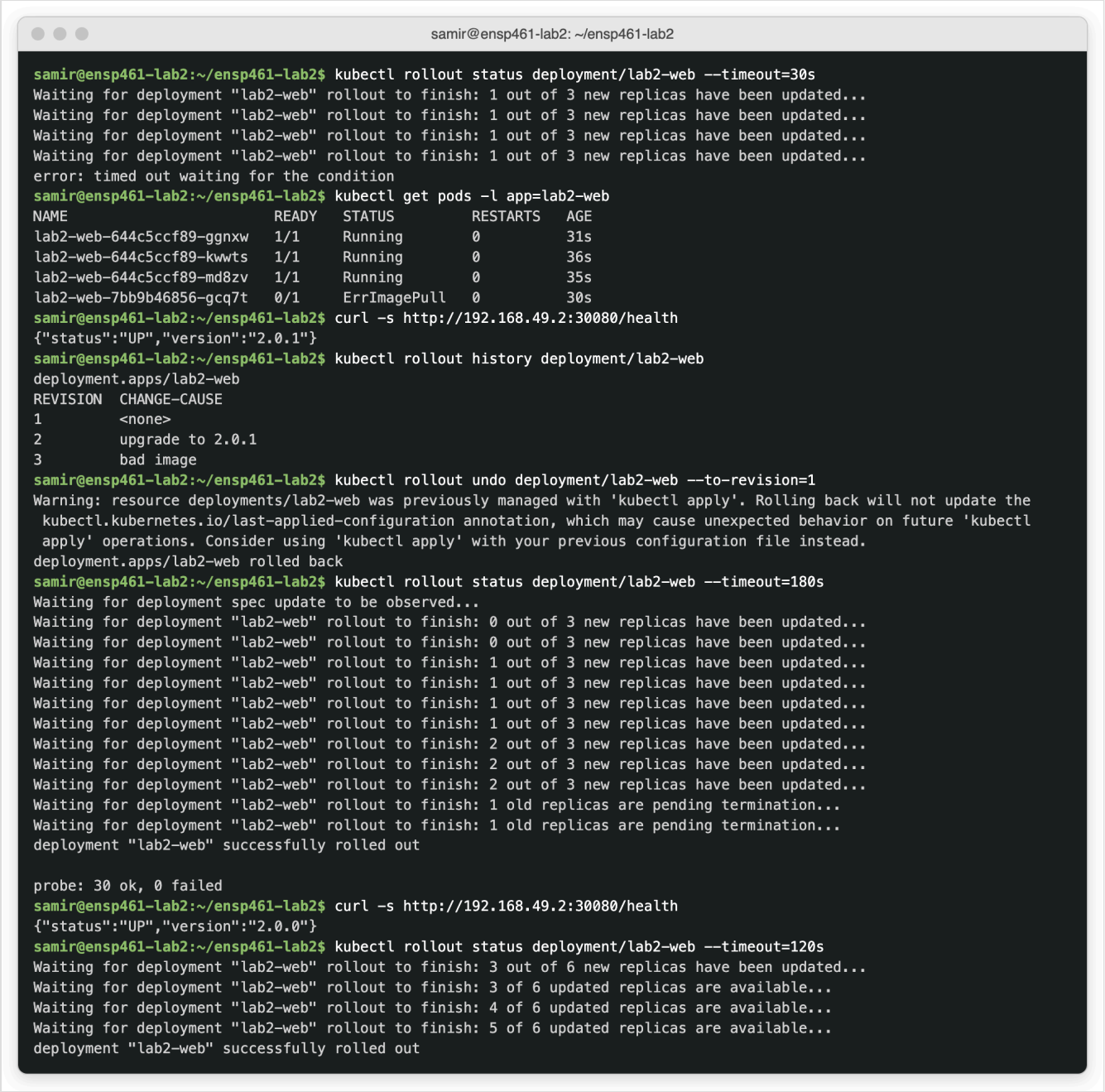


Fig: Bad image stalls the rollout without an outage; rollback to revision 1

Because `maxUnavailable` is 0, the broken pod never became ready, so no old pod was removed. All three 2.0.1 pods kept serving and users saw no outage. The rollout simply stalled until it was rolled back. `rollout undo --to-revision=1` scaled the original ReplicaSet `lab2-web-fd84f8f84` back up: a rollback reuses the old ReplicaSet instead of creating a new one.

Scaling

```
$ kubectl scale deployment/lab2-web --replicas=6
$ kubectl get deployment lab2-web
NAME       READY   UP-TO-DATE   AVAILABLE
lab2-web   6/6     6            6
$ kubectl scale deployment/lab2-web --replicas=2
$ kubectl get deployment lab2-web
lab2-web   2/2     2            2
$ kubectl scale deployment/lab2-web --replicas=3 && kubectl rollout status deployment/lab2-web
deployment "lab2-web" successfully rolled out
```

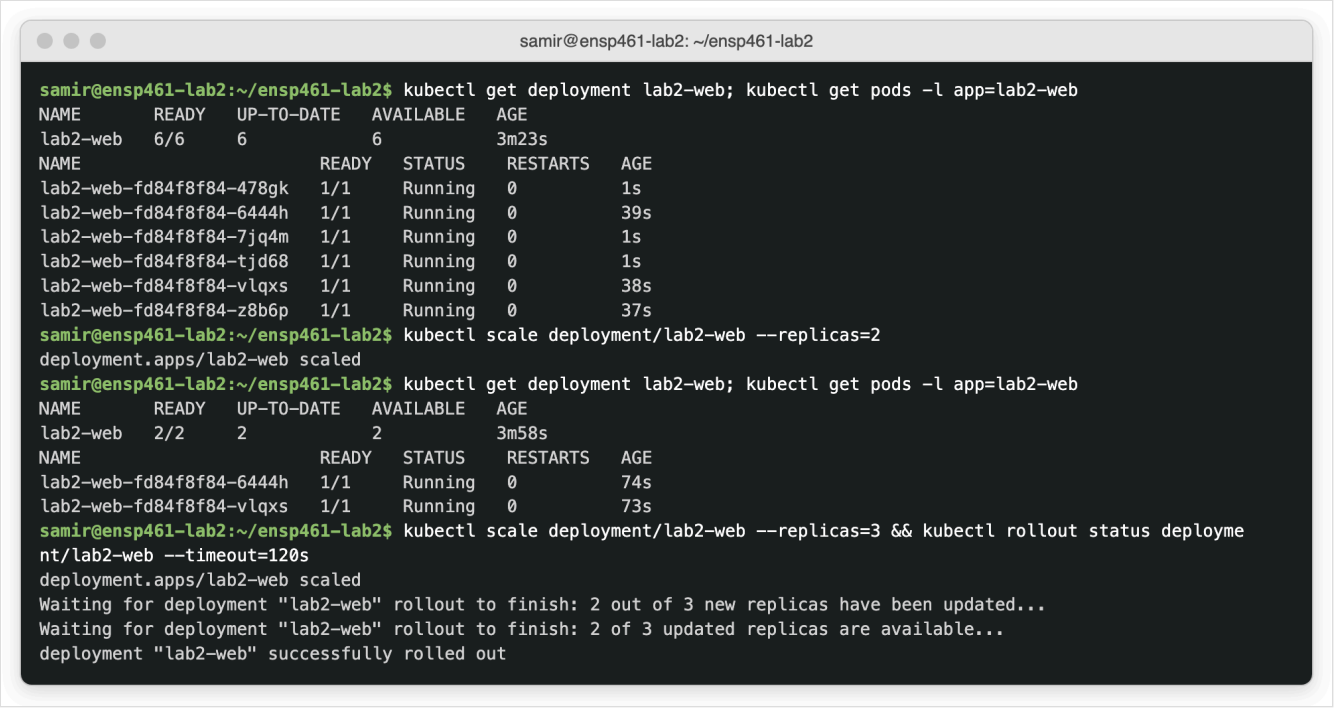


Fig: Scale out to 6, in to 2, back to 3

Scaling only changes the replica count of the current ReplicaSet, so pod names keep the same `fd84f8f84` hash. Scaling in removed the newest pods first (the four created 1 to 39 s earlier) and kept the two oldest.

Problem found: requests failed during rollouts

The first run (`docs/logs/q6-rollout-run1.log`) used the same strategy without a `preStop` hook.

Run	Rolling update	Rollback
Without `preStop`	29 ok, 5 failed	80 ok, 2 failed
With `preStop: sleep 5`	61 ok, 0 failed	30 ok, 0 failed

`maxUnavailable: 0` guarantees enough ready pods, but not that a terminating pod is out of rotation before it stops. When a pod is deleted, Kubernetes sends SIGTERM and removes its endpoint at the same time. gunicorn exited at once, while kube-proxy still routed some new connections to it for a moment. The `preStop` sleep keeps the old pod serving for 5 s while the endpoint removal spreads, and only then does gunicorn get SIGTERM.